Viva Q&A Guide — PhishGuard

Covers everything from the dissertation

Keep it natural. Don’t memorise word-for-word. Understand the reasoning and speak in your own words.

SECTION A: Topic & Motivation

Q1. Why did you choose this topic?

“Phishing is the most common cyber attack in the world — over 4.7 million attacks reported in 2023 alone. I was interested in cybersecurity and wanted to do something practical. When I looked at the literature, I noticed that blacklists are always behind and content-based methods are too slow. I thought — what if you could tell whether a URL is dangerous just by looking at the text of the link itself? When I found that no one had done a proper controlled comparison using only structural URL features on a large dataset, I knew that was my gap.”

Q2. What is the practical relevance of this research?

“If this works, it can be deployed as a browser extension, an email filter, or a mobile app. It runs in under 1 millisecond per URL, needs no internet connection for the analysis, and does not send the URL to any server — so it is also privacy-preserving. Small businesses that cannot afford enterprise security tools could use something like this as a first layer of defence.”

Q3. Who are the stakeholders for this research?

“End users who click links daily — they benefit from real-time protection. Cybersecurity vendors who could integrate a lexical classifier as a fast pre-filter before more expensive analysis. Academic researchers who can replicate and extend the work using my open-source code. And organisations — especially SMEs — that need lightweight security without expensive infrastructure.”

SECTION B: Literature

Q4. What are the key studies that influenced your work?

“Three main ones. Ma et al. 2009 proved that URL features alone have predictive power — you do not need to load the page. Sahingoz et al. 2019 showed Random Forest outperforms linear models for URL classification. And Rao and Pais 2020 used deep learning for URLs but did not report False Positive Rate — which is the gap I address.”

Q5. What is the research gap you identified?

“Three gaps. First, no study had done a controlled comparison of linear versus non-linear classifiers using only structural lexical features on a large dataset. Second, most studies do not report False Positive Rate, which is the most important metric for real-world usability. Third, most studies do not publish their code or data, making replication impossible. My study addresses all three.”

Q6. How does your work differ from Sahingoz et al.?

“They used a smaller dataset — about 73,000 URLs versus my 641,000. They also included NLP features like bag-of-words, which are computationally expensive. I deliberately restricted myself to 10 simple structural features that can be computed in

under 1 millisecond, making my approach suitable for client-side deployment. I also explicitly report FPR, which they did not."

Q7. What theoretical perspectives influenced your work?

"Three. First, Ma et al. 2009 — proving URL features alone have predictive power. Second, Breiman's 2001 paper on Random Forests — showing that ensembles of decision trees can capture non-linear patterns that single models miss. Third, the CRISP-DM framework by Chapman et al. 2000 — giving me a structured methodology from business understanding through to deployment."

Q8. What is the 'zero-hour' problem you mention?

"It is the gap between when a phishing URL is created and when it gets added to a blacklist. Sheng et al. found this takes an average of 12 hours. During those 12 hours, the URL is active and victims are being compromised. Machine learning solves this because it classifies based on patterns, not lists — so it can catch URLs it has never seen before."

SECTION C: Methodology

Q9. Why did you choose CRISP-DM?

"CRISP-DM is the industry standard for data mining projects. It has six clear phases that map naturally to what I needed to do — understand the problem, understand the data, prepare it, build models, evaluate them, and deploy. I mapped each phase to a two-week Agile sprint, which gave me structure and clear deliverables at every stage."

Q10. What is your research philosophy and why?

“Positivist. I believe knowledge comes from measurable, observable phenomena. My research question is empirical — can a model achieve certain measurable performance levels? I use objective metrics like Accuracy, F1-Score, and FPR that do not depend on my interpretation. Anyone running my code with the same data will get the same results.”

Q11. Why a deductive approach?

“Because the literature already suggests that non-linear models should outperform linear ones for this task. I had a hypothesis to test, not a pattern to discover. Deductive means going from theory to experiment — I took the theory from the literature and designed a controlled experiment to confirm or reject it.”

Q12. Why quantitative and not qualitative?

“There are no human participants in this study. No interviews, no surveys, no subjective experiences. It is purely about measuring model performance on data. Quantitative methods are the only appropriate choice. The results are numbers — accuracy percentages, F-scores, rates — and they need statistical analysis, not thematic analysis.”

Q13. How did you ensure your data collection was reliable and valid?

“I used a publicly available dataset from Kaggle with over 641,000 labelled URLs. For reliability: I removed duplicates, cleaned missing values, standardised to lowercase, and fixed the random seed to 42 so results are reproducible. For validity: I used stratified splitting to maintain class balance, and 5-fold cross-validation to confirm results are not dependent on one lucky split. The standard deviation was only 0.17% — very stable.”

Q14. Why an $^{80}/_{20}$ split?

“It is the standard in machine learning for datasets of this size. 80% gives the model over 512,000 training examples — more than enough to learn patterns. 20% gives me 128,000 test URLs — large enough for statistically reliable evaluation. I used stratified splitting so both sets keep the same $^{67}/_{33}$ class ratio.”

Q15. Why stratified splitting?

“The dataset is imbalanced — 67% benign, 33% malicious. Without stratification, a random split could accidentally put too many malicious URLs in one set. Stratification guarantees both training and test sets reflect the true class distribution, so the evaluation is fair and representative.”

Q16. Why these three classifiers specifically?

“I chose one from each algorithmic family. Logistic Regression is the standard linear baseline — simple, fast, interpretable. Random Forest is a non-linear ensemble — the literature’s recommended choice for this task. LinearSVC is another linear model to confirm whether the performance ceiling is due to the algorithm or the linearity itself. The fact that LR and SVM got almost identical results — 78.94% and 78.43% — proves it is the linearity that is the bottleneck.”

Q17. Why not deep learning or XGBoost?

“I wanted to keep the comparison focused and controlled — three clearly different algorithm types. Adding more would make the study broader but shallower. Deep learning also needs much more data and compute, and is harder to interpret. XGBoost is a valid future direction, which I recommend in Chapter 6. But for answering my specific research question — linear vs non-linear on lexical features — three classifiers is sufficient.”

SECTION D: Implementation

Q18. How did you handle malformed URLs?

“Many malicious URLs are deliberately broken — they crash Python’s standard urlparse library. I avoided urlparse entirely and used vectorised pandas string operations instead. I also wrote custom exception handling so that if a URL cannot be parsed, the pipeline logs it and skips it rather than crashing. This is important because in a real deployment, you will encounter broken URLs constantly.”

Q19. Why did you choose these 10 specific features?

“They are all structural properties of the URL string that can be computed instantly without any external calls. Each one has a clear rationale from the literature. For example, dot_count captures subdomain stacking — attackers add dots to make URLs look legitimate. Path_length captures directory obfuscation. Entropy captures randomness — algorithmically generated URLs have high entropy. Together, these 10 features cover the main manipulation techniques attackers use.”

Q20. Why vectorised extraction instead of urlparse?

“Two reasons. First, performance — vectorised pandas operations process 641,000 URLs in seconds, while a Python loop with urlparse would take much longer. Second, robustness — urlparse throws exceptions on malformed URLs, which are common in malicious datasets. My approach handles any string without crashing.”

Q21. Explain the Random Forest hyperparameters you chose.

“200 trees — the literature shows performance levels off between 100 and 300 trees for datasets this size. Max depth of 20 — deep enough to learn complex patterns but not so deep that trees memorise individual training examples. Min samples leaf of 5 — each terminal node must have at least 5 examples, which prevents overfitting to noise. I confirmed these through preliminary experiments.”

Q22. Why class_weight=‘balanced’ instead of SMOTE?

“Both address class imbalance, but class_weight=‘balanced’ is simpler — it adjusts the loss function to penalise misclassification of the minority class more heavily. SMOTE creates synthetic data points, which can introduce noise and may not represent real phishing patterns. Since I already had over 200,000 malicious URLs, I did not need synthetic examples.”

Q23. How did you ensure reproducibility?

“Three things. Fixed random seed of 42 for all splits and model training. All code published on GitHub with modular scripts. And automated evidence generation — every sprint produces PNG plots and text summaries that are committed to the repository. Anyone can clone the repo, run the scripts, and get the same results.”

SECTION E: Results & Findings

Q24. What are your main findings?

“Random Forest achieved 94.03% accuracy, F1-Score of 0.9120, and False Positive Rate of 5.45%. The linear models — Logistic Regression and SVM — both scored around 78-79% accuracy with recall of only 50%, meaning they missed half of all malicious URLs. The 15-percentage-point gap proves that the relationship between URL features and phishing is non-linear.”

Q25. What does the 5.45% FPR mean in practice?

“Out of every 100 legitimate URLs a user visits, about 5 would be incorrectly flagged as suspicious. That is roughly 1 in 18. For a first-layer filter that would be combined with other security measures, this is acceptable. It meets my non-functional requirement of FPR below 10%.”

Q26. Why did the linear models perform so poorly?

“Because phishing patterns are non-linear. A URL with many dots is not necessarily malicious — bbc.co.uk has three dots. But a URL with many dots AND a long path AND high entropy is very likely malicious. Linear models try to find one straight line through the feature space — they cannot capture these interactions. Random Forest builds 200 trees that each split on different features, so it can model complex combinations.”

Q27. Why did LR and SVM get almost the same accuracy?

“Because they are both linear classifiers — they just use different loss functions and optimisation methods. The fact that they converge to the same accuracy — 78.94% vs 78.43% — proves that the limitation is the linear assumption itself, not the specific algorithm. This is actually an important finding: tuning different linear algorithms for this task is pointless. You need to go non-linear.”

Q28. Explain the cross-validation results.

“I ran 5-fold stratified cross-validation on a balanced subsample of 100,000 URLs. Random Forest scored 91.95% accuracy with a standard deviation of only 0.17%. This means the results are extremely stable — not dependent on one lucky data split. The slightly lower accuracy compared to the full test set — 91.95% vs 94.03% — is expected because the balanced subsample removes the natural class distribution advantage.”

Q29. What does ROC AUC of 0.9855 mean?

“If you randomly pick one malicious URL and one benign URL, the model will correctly rank the malicious one as more suspicious 98.55% of the time. A perfect model scores 1.0, a random guess scores 0.5. So 0.9855 shows the model has excellent discriminative power across all possible classification thresholds.”

Q30. What are the top features and why?

“Dot_count is number one with an F-score of 31,994 — because subdomain stacking is the most common phishing technique. Path_length is second at 19,041 — because attackers use deep directory structures to hide the true destination. Slash_count is third at 10,162 — closely related to path complexity. These align perfectly with known attacker tactics described in the cybersecurity literature.”

Q31. How do your results compare to the literature?

“Sahingoz et al. reported 97.3% accuracy but used a smaller dataset and additional NLP features. Ma et al. reported 95-99% but combined lexical and host-based features. For a purely lexical approach on 641,000 URLs, my 94.03% is competitive. The key difference is that I provide a controlled comparison with explicit FPR reporting — which most studies do not do.”

SECTION F: Discussion & Critical Thinking

Q32. What is the biggest weakness of your study?

“Temporal drift. The dataset is historical — phishing tactics evolve. A model trained on 2024 data may not perform as well on 2026 URLs if attackers have changed their patterns. A production system would need periodic retraining with fresh data. I acknowledge this honestly in the dissertation.”

Q33. Could an attacker evade your system?

“Yes. A sophisticated attacker who knows the model is lexical-only could register a short, clean domain name that scores low on all 10 features. This is the ‘perfect mimicry’ limitation. However, this system is designed as a first-layer filter, not a standalone solution. In a multi-layered architecture, URLs that pass the lexical check would still be analysed by content-based or reputation-based systems.”

Q34. Why is FPR more important than accuracy for deployment?

“Because if your system blocks too many legitimate websites, users will turn it off. A system with 99% accuracy but 20% FPR would block 1 in 5 legitimate sites — no one would tolerate that. My 5.45% FPR means users can browse normally with minimal disruption, while still catching 93% of malicious URLs.”

Q35. What are the implications for business practice?

“Organisations can use a lexical Random Forest classifier as a fast pre-filter. It costs nothing to run — no API calls, no infrastructure. It can be embedded in a browser extension, email gateway, or mobile app. Cybersecurity vendors could use it as the first layer that quickly filters obviously suspicious URLs before passing the rest to more expensive analysis systems.”

SECTION G: Limitations & Future Work

Q36. What are the main limitations?

“Four. First, temporal drift — the model may degrade over time. Second, lexical features alone cannot catch perfect mimicry. Third, I only compared three algorithms — not gradient boosting or deep learning. Fourth, the 5.45% FPR is acceptable but not negligible — in high-traffic environments, that is still a lot of false alarms.”

Q37. What would you do differently if you repeated the study?

“Three things. First, temporal validation — train on older URLs, test on newer ones, to measure how fast the model degrades. Second, add character-level n-grams to catch brand spoofing like ‘paypa1’ instead of ‘paypal’. Third, test XGBoost or LightGBM, which the literature suggests might outperform Random Forest for tabular data.”

Q38. What do you recommend for future research?

“Five directions. Lightweight NLP features like n-grams. A temporal retraining pipeline with real-time threat intelligence. Browser extension deployment with user experience testing.

Gradient boosting methods. And multi-class classification — distinguishing phishing from malware from defacement, not just benign versus malicious."

SECTION H: The Application (PhishGuard)

Q39. How does PhishGuard work technically?

"It is built with React. When a user submits a URL, JavaScript extracts the same 10 lexical features used in the research — url_length, dot_count, path_length, entropy, and so on. These are passed to a scoring function that replicates the Random Forest model's decision logic. The risk score from 0 to 100 is displayed with a colour-coded verdict and a breakdown of each feature's contribution. Everything runs in the browser — no server needed."

Q40. Why did you build a web application?

"To demonstrate real-world deployment viability. A Python script proves the model works in a lab. A web application proves it can work in practice — in a browser, in real time, with a user interface that non-technical people can understand. It also satisfies the Deployment phase of CRISP-DM and shows the practical contribution of the research."

Q41. Why client-side and not server-side?

"Privacy and speed. If the URL is sent to a server, you introduce network latency and the user's browsing history is exposed. Client-side means the analysis happens instantly in the browser and no data leaves the user's device. This is a key advantage of lexical features — they are simple enough to compute in JavaScript."

SECTION I: Ethics & Project Management

Q42. What ethical considerations did you address?

“The project was approved as low risk. No human participants, no personal data. All URLs were treated as text strings in an offline Python environment — I never visited any malicious URLs. The dataset is publicly available on Kaggle. I followed the NCG Research Ethics Policy and submitted the HE35 form in December 2025.”

Q43. How did you manage the project?

“Agile Scrum with two-week sprints, tracked on Trello. Each sprint had clear deliverables — a working increment committed to GitHub. I had regular supervisor meetings documented in the logbook. The CRISP-DM phases mapped to sprints gave me structure, and the retrospective cards on Trello helped me identify and fix bottlenecks early.”

Q44. What were the main challenges you faced?

“Three. First, malformed URLs crashing the parser — solved with custom exception handling and vectorised operations. Second, class imbalance — solved with balanced class weights. Third, computational cost of cross-validation on 641,000 URLs — solved with balanced subsampling of 100,000 rows, which is standard practice.”

SECTION J: Personal Reflection

Q45. What did you learn from this project?

“Technically — that empirical testing matters more than theoretical assumptions. I expected the linear models to be decent, but they were surprisingly poor. That taught me to always test rather than assume. In terms of project management — Agile sprints kept me on track and prevented scope creep. And personally — I learned to embrace negative results. Documenting why the linear models failed was actually one of the most informative parts of the dissertation.”

Q46. What are you most proud of?

“Two things. First, that the research question has a clear, definitive answer — yes, lexical features work, but only with non-linear models. Second, that I built a working prototype that anyone can use right now. It is not just theory — it is a real tool that demonstrates the research in practice.”

Quick Numbers Reference

What	Number
Dataset size	641,053 URLs
Split	$^{80}/_{20}$ stratified
Class balance	67% benign, 33% malicious
RF Accuracy	94.03%
RF F1-Score	0.9120
RF FPR	5.45%
RF ROC AUC	0.9855
LR Accuracy	78.94%
SVM Accuracy	78.43%
Top feature	dot_count (F = 31,994)
Number of features	10
Number of trees in RF	200
Max depth	20
Cross-val accuracy	91.95% ± 0.17%
Training time (RF)	~41 seconds
Feature extraction	< 1ms per URL
Random seed	42